

# docs\_chain

**Aucun document suivi ne peut se désynchroniser — hachage de contenu, bidirectionnel, atomique.**

## Résumé

Docs Chain est un moteur universel de propagation des dépendances entre documents et bases de données, implémenté en Go et bidirectionnel. Lorsqu'un élément d'une chaîne enregistrée est modifié — source Markdown, export HTML/PDF/DOCX ou base de données SQLite —, il détecte ce changement par hachage de contenu et le propage de manière atomique à tous les éléments connectés.

## Description courte

Un moteur Go assurant la synchronisation entre documents et bases de données. Grâce à une recomputation incrémentale à la Salsa, basée sur le hachage de contenu et appliquée à un DAG (ordonnancement topologique de Kahn, coupure précoce, arêtes de synchronisation bidirectionnelles, renommage atomique + commits transactionnels SQLite), il régénère les exports dès qu'un artefact lié est modifié.

## Description longue

Docs Chain est ce que l'on construit après avoir écrit une fois de trop le même script shell fragile du type « *régénérer le PDF quand le Markdown change* ». Il remplace toute cette catégorie de solutions artisanales de synchronisation par un véritable moteur. Il modélise les documents et bases de données d'un projet comme les maillons d'une chaîne et, dès qu'un maillon est modifié, propage ce changement à tous les éléments connectés, dans toutes les directions déclarées — régénérant et réexportant de manière atomique pour garantir qu'aucun artefact suivi ne puisse jamais se désynchroniser. Sa conception emprunte sa rigueur au monde des systèmes de compilation incrémentale plutôt qu'à celui des scripts : la détection des changements repose sur le **hachage**

**de contenu, et non sur la date de modification**, si bien qu'un touch ne déclenche rien, tandis qu'une modification d'un seul octet entraîne exactement les recompilations nécessaires — sans fausses alertes ni changements manqués. Formulé en une seule phrase, il s'agit d'une recomputation incrémentale à la Salsa, basée sur le hachage de contenu et appliquée à un DAG, avec un ordonnancement topologique de Kahn, une coupure précoce éliminant les sous-arbres inchangés, des arêtes de synchronisation bidirectionnelles à autorité déclarée, ainsi que des renommages atomiques et des commits transactionnels SQLite, de sorte qu'un plantage en cours de propagation ne laisse jamais un export à moitié écrit. Il est distribué sous forme de sous-module vasic-digital et intégré comme composant central du sous-module HelixConstitution, ce qui permet à tout projet adoptant la constitution de bénéficier de Docs Chain dès le départ et d'enregistrer ses propres chaînes via des YAML spécifiques au contexte. L'implémentation affiche clairement son état (conformément à la constitution §11.4.6) : les phases 1 à 4 (DAG central + hachage, adaptateurs/transmutations de nœuds, orchestrateur de propagation avec atomicité, CLI multi-contextes piloté par configuration avec sync/verify/doctor/graph/watch) sont implémentées et testées ; une phase 4b ajoute des modules intégrés génériques de synchronisation bidirectionnelle md-to-sqlite/sqlite-to-md (pure-Go, dérive au niveau des lignes, aller-retour stable au niveau des octets) et un module intégré colorize-html ; la phase 5, couvrant des tests e2e réels complets, est implémentée et VALIDÉE. Les phases 6 et 7 (distribution via la constitution, intégration ATMOSphere) restent PLANIFIÉES et soumises à un contrôle opérateur. Herald est le premier consommateur aval réel, synchronisant un corpus multi-formats de 66 documents avec une vérification propre.

## **Pourquoi nous l'avons conçu**

La documentation, les exports et les bases de données divergent dès qu'ils sont maintenus manuellement ou via des scripts fragiles. Docs Chain rend la synchronisation mécanique, précise grâce au hachage de contenu et atomique, de sorte qu'une modification en un point quelconque d'une chaîne met correctement et en toute sécurité à jour tout ce qui en dépend (en aval comme en amont).

Contenu

## **Pourquoi c'est révolutionnaire**

Il prend les garanties de fiabilité, durement acquises et tenues pour acquises par les auteurs de compilateurs et de systèmes de build — graphes de dépendances à hachage de contenu,

recomputation minimale, commits atomiques — et les applique à la documentation et aux bases de données, un domaine qui, historiquement, se traînait péniblement avec des tâches cron et de bonnes intentions. Une synchronisation bidirectionnelle véritable signifie que la relation entre une source et son export est imposée dans les deux sens : « *la documentation est obsolète* » et « *l'export ne correspond pas à la source* » cessent d'être des bugs récurrents pour devenir des états que le moteur refuse tout simplement d'autoriser.

## Ce qui est innovant

- Recomputation incrémentale par hachage de contenu (et non par horodatage) sur un DAG avec coupure précoce.
- Synchronisation bidirectionnelle avec déclaration d'autorité (docs ↔ exports ↔ SQLite).
- Commit atomique par renommage + transaction SQLite pour une propagation résistante aux plantages.
- Tour de passe-passe md-to-sqlite/sqlite-to-md en pur Go avec détection de dérive au niveau des lignes.

## Défis et solutions

- **Reconstructions intempestives** : résolues par la détection par hachage de contenu plutôt que par horodatage.
- **Mises à jour partielles/corrompues** : résolues par le renommage atomique et les transactions SQLite.
- **Ordonnancement correct des membres multiples** : résolu par l'ordonnancement topologique de Kahn avec coupure précoce.
- **Rapport de capacités honnête** : résolu en marquant chaque phase comme IMPLEMENTÉE ou PLANIFIÉE selon le §11.4.6.

## Pile technologique (pourquoi et comment)

- **Go** — moteur complet (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs\_chain).
- **DAG + tri topologique de Kahn** — ordonnancement des dépendances avec coupure précoce.
- **SQLite (modernc en pur Go)** — membres de la base de données et commits transactionnels.
- **fsnotify** — démon watch pour une propagation en temps réel.
- **Configuration YAML** — enregistrement des chaînes par contexte.

- **exec : transformations** — génération pluggable  
Markdown→HTML/PDF/DOCX.

Transparence de la feuille de route : les phases 6-7  
(distribution de la constitution, intégration ATMOSphere)  
sont PLANIFIÉES / soumises à validation par l'opérateur —  
non livrées.